

# AI BlogNest

AI-Powered Blog Management REST API — Project Documentation

Node.js • Express.js • PostgreSQL • JWT Authentication • Google Gemini AI

## Description

---

The AI BlogNest backend is a secure and intelligent REST API platform designed to simplify user authentication, blog creation, blog management, and AI-assisted content generation.

The application is built using Node.js and Express.js and uses PostgreSQL as its primary relational database. The PostgreSQL database is accessed through the node-postgres (pg) library, providing efficient database connectivity and SQL-based data management.

The platform integrates Google Gemini AI to provide intelligent content-generation features such as AI-powered blog generation and AI-powered blog summarization. Users can provide a topic and category to generate blog content automatically, while existing content can also be summarized using the AI service.

Security is implemented using JSON Web Tokens (JWT) for authentication and bcrypt.js for secure password hashing. Protected routes require a valid JWT authentication token before allowing users to perform operations such as creating, updating, deleting, or generating blogs.

The backend follows a modular architecture by separating routes, controllers, middleware, database configuration, and AI services. This makes the application easier to maintain, test, and extend.

The system provides complete CRUD operations for blogs along with user registration, login, profile access, AI blog generation, and AI summarization.

## Scenario-Based Case Study

---

### Background

A content creator wants to create and manage blog posts efficiently while also reducing the time required to write and summarize content manually.

### Problem

- Creating blog content manually can be time-consuming.
- Users need a secure authentication system.
- Blog posts need to be stored and managed efficiently.
- Users need the ability to create, view, update, and delete their own blog posts.
- Large blog content can be difficult to summarize manually.
- A centralized backend API is required to connect these functions with a frontend application or API testing tool.

### Solution

The AI BlogNest backend provides a centralized REST API for authentication, blog management, database operations, and AI-assisted content generation.

The application uses Node.js and Express.js for server-side processing and PostgreSQL for persistent storage.

JWT authentication protects user-specific operations, while bcrypt.js securely hashes passwords before storing them in the database.

Google Gemini AI is integrated through a dedicated service layer to provide automatic blog generation and summarization.

### Usage

#### User Usage

A registered user can:

- Create an account.
- Log in using their email and password.
- Access their authenticated profile.
- Create blog posts.
- View all available blogs.
- View an individual blog.
- Update their own blog.
- Delete their own blog.
- Generate a blog using Gemini AI.
- Summarize existing content using Gemini AI.

#### API Usage

The API can be tested using Postman by sending HTTP requests to the Express server.

The backend provides endpoints for authentication, blog management, and AI services.

### Outcome

The AI BlogNest backend provides a centralized platform for secure blog management and AI-assisted content creation.

PostgreSQL maintains structured user and blog data, JWT protects authenticated resources, and Gemini AI reduces the effort required to generate and summarize blog content.

## 1. Software Requirements

---

- **Operating System:** Windows 10/11, macOS, or Linux.
- **Node.js:** Runtime environment used to execute the backend application.
- **npm:** Package manager used to install and manage Node.js dependencies.
- **Express.js:** Web framework used to create REST API routes and handle HTTP requests.
- **PostgreSQL:** Relational database used to store users and blog posts.
- **node-postgres (pg):** PostgreSQL client library used by Node.js to communicate with the database.
- **Google Gemini API:** External AI service used for blog generation and summarization.
- **JWT:** Used for authentication and protected API access.
- **bcrypt.js:** Used for secure password hashing.
- **Postman:** Used for API testing.
- **Visual Studio Code:** Development environment used to write and manage the project.

## 2. Hardware Requirements

---

- **Processor:** Intel Core i5 / AMD Ryzen 5 or equivalent.
- **RAM:** 8 GB minimum.
- **Storage:** At least 1 GB of available space for the project, Node.js packages, PostgreSQL and development files.
- **Internet Connection:** Required for communication with the Google Gemini API.

## EPIC 1: PROJECT ARCHITECTURE

The AI BlogNest application follows a modular backend architecture. Incoming HTTP requests are received by the Express server and passed through the appropriate route, middleware, and controller before interacting with PostgreSQL or the Gemini AI service.

### Backend Architecture Diagram

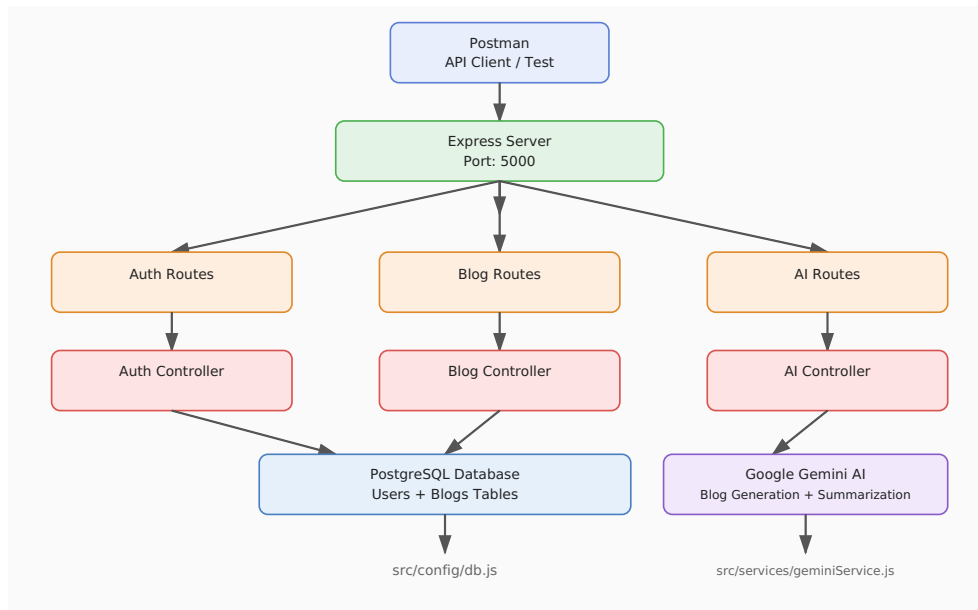


Fig 1.1 — AI BlogNest backend architecture diagram

### Components Definition

- **Client / API Interface:** Postman is used as the client interface for sending HTTP requests to the backend. Requests contain the HTTP method, URL, request headers, JSON request body, and an authorization token where required. The API returns structured JSON responses.
- **Express Server:** The main gateway of the application. It receives HTTP requests, parses JSON bodies, enables CORS, logs requests using Morgan, maps requests to API routes, and sends JSON responses. The server runs on `http://localhost:5000`.
- **Authentication Middleware:** Verifies JWT tokens sent through the `Authorization` header in the format `Bearer <JWT_TOKEN>`, validates the token using the configured JWT secret, and retrieves the corresponding user from PostgreSQL.
- **Controller Layer:** Contains the main application logic, split across `authController.js`, `blogController.js`, and `aiController.js`, handling registration, login, profile access, blog CRUD, AI generation and summarization.
- **Database Layer:** Uses the PostgreSQL `pg` library. The connection is configured in `src/config/db.js`, using a PostgreSQL connection pool to execute SQL queries.
- **AI Service Layer:** The Gemini integration is separated into `src/services/geminiService.js`, which communicates with Google Gemini for blog generation and summarization.

### Entity-Relationship Diagram

The application uses a relational PostgreSQL database containing two main tables: `users` and `blogs`, linked in a one-to-many relationship.

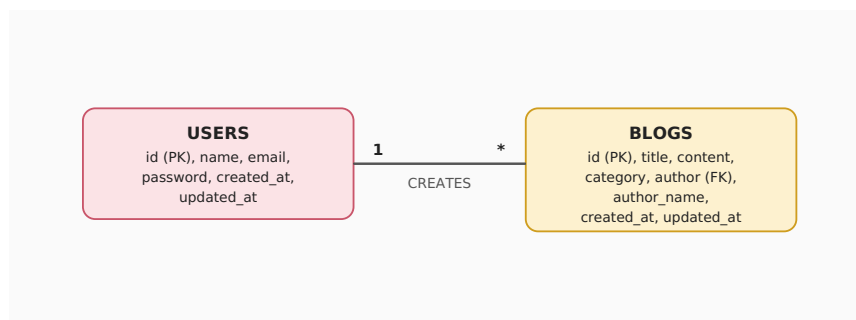


Fig 1.2 — AI BlogNest entity-relationship diagram

### 1. User Entity

The `users` table stores registered user information.

| Field      | Data Type    | Description           |
|------------|--------------|-----------------------|
| id         | SERIAL       | Primary key           |
| name       | VARCHAR(100) | User name             |
| email      | VARCHAR(255) | Unique email address  |
| password   | VARCHAR(255) | Hashed password       |
| created_at | TIMESTAMP    | Account creation time |
| updated_at | TIMESTAMP    | Account update time   |

**Primary Key:** id    **Constraint:** email UNIQUE

2. Blog Entity

The `blogs` table stores blog posts created by users.

| Field       | Data Type    | Description                   |
|-------------|--------------|-------------------------------|
| id          | SERIAL       | Primary key                   |
| title       | VARCHAR(255) | Blog title                    |
| content     | TEXT         | Blog content                  |
| category    | VARCHAR(100) | Blog category                 |
| author      | INTEGER      | Foreign key referencing users |
| author_name | VARCHAR(100) | Author name                   |
| created_at  | TIMESTAMP    | Blog creation time            |
| updated_at  | TIMESTAMP    | Blog update time              |

Relationship

A single user can create multiple blogs: `User 1 — * Blogs`. The relationship is implemented using `blogs.author → users.id` with a foreign key constraint ( `ON DELETE CASCADE` ).

Key Structural Mappings

**User to Blog** — One-to-Many relationship. One user can create many blog posts.

**Blog to User** — Each blog belongs to one authenticated user.

Key Features

- 1. **User Registration** — Users register using name, email, and password. Passwords are hashed using `bcrypt.js` before being stored in PostgreSQL.
- 2. **User Login** — Registered users log in using their email and password. After successful authentication, the server generates a JWT token used to access protected resources.
- 3. **JWT Authentication** — Protected routes require a valid JWT token, verified against the configured secret before the corresponding user is retrieved from PostgreSQL.
- 4. **Blog CRUD Operations** — Users can create a new blog, retrieve all blogs, retrieve an individual blog by ID, update their own blog, and delete their own blog.
- 5. **AI Blog Generation** — A user provides a topic and category; the AI service generates blog content based on the topic and the generated blog is stored in PostgreSQL.
- 6. **AI Blog Summarization** — The system summarizes blog content using Gemini AI. The user provides existing content, which is sent to Gemini, and the generated summary is returned as JSON.
- 7. **Input Validation** — The application uses `express-validator` to validate incoming requests (name provided, valid email, minimum password length, required blog title/content/category).
- 8. **Error Handling** — A centralized error middleware at `src/middleware/errorMiddleware.js` handles application errors and returns JSON responses.
- 9. **CORS Support** — CORS middleware allows communication between the backend API and external clients or frontend applications.
- 10. **Request Logging** — Morgan is used to log HTTP requests and responses during development.

Roles and Responsibilities

The current version of AI BlogNest does not implement separate Admin, Editor, Author, or Reader role permissions. Instead, the current authentication model is:

**Registered User** — can log in, access their profile, create blogs, view blogs, update their own blogs, delete their own blogs, and generate AI blogs.

**Public User / Unauthenticated Request** — public endpoints allow users to view blogs, view an individual blog, and use the AI summarization endpoint according to the current route configuration.

## User Flow

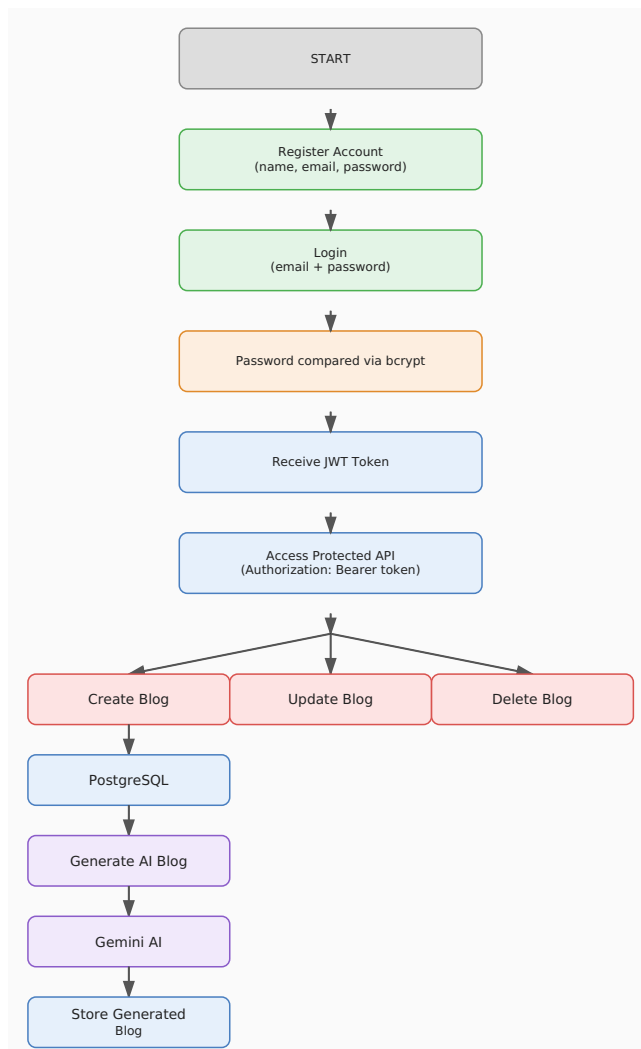


Fig 1.3 — AI BlogNest user flow diagram

## MVC / Layered Architecture Pattern

The project follows a controller-based layered backend architecture. Because AI BlogNest is a REST API and does not render HTML pages, there is no traditional frontend View layer inside this backend.

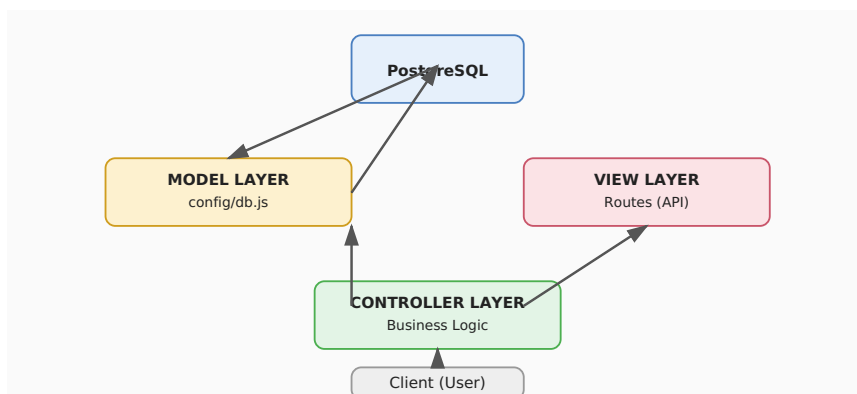


Fig 1.4 — Layered architecture pattern

## Configuration Layer

The configuration layer manages external services such as PostgreSQL. File: `src/config/db.js`. It creates the PostgreSQL connection pool.

## Controller Layer

The controller layer contains the main business logic:

- **authController.js** — Responsible for registration, login, profile retrieval.
- **blogController.js** — Responsible for creating blogs, getting all blogs, getting a blog by ID, updating blogs, deleting blogs.
- **aiController.js** — Responsible for AI blog generation, AI summarization.

## Service Layer

The service layer contains external AI integration logic. **geminiService.js** is responsible for communicating with Google Gemini AI. It provides `callGemini()`, which sends prompts to Gemini and returns generated text.

## Middleware Layer

The project contains middleware for authentication and error handling:

- **authMiddleware.js** — Validates JWT authentication tokens and retrieves the authenticated user from PostgreSQL.
- **errorMiddleware.js** — Handles application errors and returns standardized JSON error responses.

## Route Layer

Routes connect HTTP requests to controllers: `src/routes/authRoutes.js`, `src/routes/blogRoutes.js`, `src/routes/aiRoutes.js`.

## Project Folder Structure

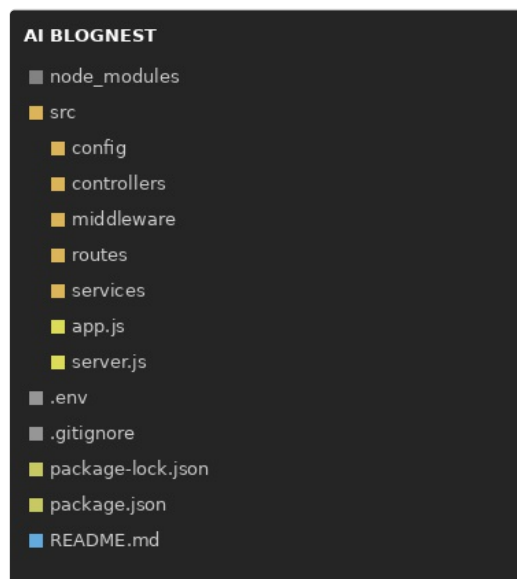


Fig 1.5 — Project folder structure in VS Code

## EPIC 2: PROJECT SETUP AND CONFIGURATION

### Creating Project Folder

---

1. Create a project folder named **AI BlogNest**.
2. Open the project folder in Visual Studio Code.
3. Open the integrated terminal.
4. Initialize the Node.js project:

```
npm init -y
```

### Installing Dependencies

---

The following dependencies are used by the project:

```
npm install express cors morgan dotenv express-validator bcryptjs jsonwebtoken pg @google/generative-ai axios
```

For development:

```
npm install nodemon --save-dev
```

### Package Configuration

---

The project uses the following scripts:

```
"scripts": {  
  "start": "node src/server.js",  
  "dev": "nodemon src/server.js"  
}
```

The development server can be started using:

```
npm run dev
```



## EPIC 3: BACKEND DEVELOPMENT

### Server.js

The `server.js` file is the entry point of the application. It loads environment variables, loads the Express application, initializes the PostgreSQL connection, and starts the server on port 5000.

```
server.js
src > server.js > ...
1  const dotenv = require('dotenv');
2  const app = require('./app');
3  const { pool, connectDB } = require('./config/db');
4
5  dotenv.config();
6  connectDB();
7
8  const PORT = process.env.PORT || 5000;
9  app.listen(PORT, () => {
10     console.log(`Server running on port ${PORT}`);
11 });
12
```

Fig 3.1 — src/server.js

### App.js

The `app.js` file configures the Express application. It initializes Express, CORS, JSON parsing, Morgan logging, authentication routes, blog routes, AI routes, and error handling.

```
app.js
src > app.js > ...
1  const express = require('express');
2  const cors = require('cors');
3  const morgan = require('morgan');
4  const authRoutes = require('./routes/authRoutes');
5  const blogRoutes = require('./routes/blogRoutes');
6  const aiRoutes = require('./routes/aiRoutes');
7  const errorMiddleware = require('./middleware/errorMiddleware');
8
9  const app = express();
10
11  app.use(cors());
12  app.use(express.json());
13  app.use(morgan('dev'));
14
15  app.use('/api/auth', authRoutes);
16  app.use('/api/blogs', blogRoutes);
17  app.use('/api/ai', aiRoutes);
18
19  app.use(errorMiddleware);
20
21  app.get('/', (req, res) => {
22     res.json({ message: 'AI BlogNest API is running' });
23 });
24
25  module.exports = app;
26
```

Fig 3.2 — src/app.js

### API Base Routes

- `/api/auth`
- `/api/blogs`
- `/api/ai`

### Expected Output

```
Server running on port 5000  
PostgreSQL connected successfully
```

## EPIC 4: DATABASE CONFIGURATION

### PostgreSQL Database

The original project used MongoDB/Mongoose. For the final AI BlogNest implementation, the database was migrated to PostgreSQL. The PostgreSQL database used by the project is:

- **Database Name:** blognest
- **Host:** localhost
- **Port:** 5432

The Node.js application connects to PostgreSQL using the `pg` package.

### Database Connection

The database connection is configured in `src/config/db.js`. The application creates a PostgreSQL connection pool. The connection is verified using `SELECT NOW();`

```
db.js
src > config > db.js >
1  const { Pool } = require('pg');
2
3  const pool = new Pool({
4    connectionString: process.env.DATABASE_URL,
5  });
6
7  const connectDB = async () => {
8    try {
9      await pool.query('SELECT NOW();');
10     console.log('PostgreSQL connected successfully');
11   } catch (error) {
12     console.error(`PostgreSQL connection error: ${error.message}`);
13     process.exit(1);
14   }
15 };
16
17 module.exports = { pool, connectDB };
18
```

Fig 4.1 — src/config/db.js

```
PostgreSQL connected successfully
```

### Database Tables

The application contains two main tables: `users` and `blogs`.

#### Users Table

```
users_table.sql
src > config > schema.sql > ...
1  CREATE TABLE users (
2    id SERIAL PRIMARY KEY,
3    name VARCHAR(100) NOT NULL,
4    email VARCHAR(255) UNIQUE NOT NULL,
5    password VARCHAR(255) NOT NULL,
6    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
7    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
8  );
9
```

Fig 4.2 — users table definition

## Blogs Table

```
blogs_table.sql
src > config > schema.sql >
1 CREATE TABLE blogs (
2     id SERIAL PRIMARY KEY,
3     title VARCHAR(255) NOT NULL,
4     content TEXT NOT NULL,
5     category VARCHAR(100) NOT NULL,
6     author INTEGER NOT NULL,
7     author_name VARCHAR(100) NOT NULL,
8     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
9     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
10
11     CONSTRAINT fk_blog_author
12         FOREIGN KEY (author)
13         REFERENCES users(id)
14         ON DELETE CASCADE
15 );
16
```

Fig 4.3 — blogs table definition

## Database Relationship

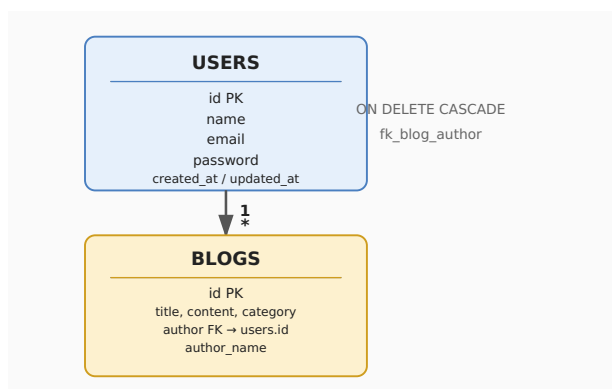


Fig 4.4 — users - blogs relationship

## EPIC 5: PROJECT EXECUTION

### Installing Postman

Postman is used to test all REST API endpoints. Start the backend using:

```
npm run dev
```

Expected:

```
Server running on port 5000
PostgreSQL connected successfully
```

### API Testing

#### 1. User Registration

**POST** `http://localhost:5000/api/auth/register`

|                                                                                                             |                                                                                                                                                     |             |
|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <b>POST</b>                                                                                                 | <code>http://localhost:5000/api/auth/register</code>                                                                                                | <b>Send</b> |
| Body                                                                                                        | Status: 201 Created Time: 140 ms Size: 397 B                                                                                                        |             |
| <pre>1 { 2   "name": "Test User", 3   "email": "testuser@gmail.com", 4   "password": "test123456" 5 }</pre> | <pre>{   "message": "User registered successfully",   "user": {     "id": 1,     "name": "Test User",     "email": "testuser@gmail.com"   } }</pre> |             |

#### 2. User Login

**POST** `http://localhost:5000/api/auth/login`

|                                                                                    |                                                                                                 |             |
|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|-------------|
| <b>POST</b>                                                                        | <code>http://localhost:5000/api/auth/login</code>                                               | <b>Send</b> |
| Body                                                                               | Status: 200 OK Time: 146 ms Size: 451 B                                                         |             |
| <pre>1 { 2   "email": "testuser@gmail.com", 3   "password": "test123456" 4 }</pre> | <pre>{   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI66MX0.R7cdn0D0_ev8Ss57k" }</pre> |             |
| Do not include your actual JWT token in the documentation.                         |                                                                                                 |             |

Do not include your actual JWT token in the documentation.

#### 3. User Profile

**GET** `http://localhost:5000/api/auth/profile` — Authorization: Bearer Token

|            |                                                                                  |             |
|------------|----------------------------------------------------------------------------------|-------------|
| <b>GET</b> | <code>http://localhost:5000/api/auth/profile</code>                              | <b>Send</b> |
| Body       | Status: 200 OK Time: 64 ms Size: 118 B                                           |             |
| (no body)  | <pre>{   "id": 1,   "name": "Test User",   "email": "testuser@gmail.com" }</pre> |             |

#### 4. Create Blog

**POST** http://localhost:5000/api/blogs — Authorization: Bearer Token

|                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                          |             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <b>POST</b>                                                                                                                                                                             | http://localhost:5000/api/blogs                                                                                                                                                                                                                                                                                          | <b>Send</b> |
| Body                                                                                                                                                                                    | Status: 201 Created Time: 59 ms Size: 541 B                                                                                                                                                                                                                                                                              |             |
| <pre>1 { 2   "title": "My First PostgreSQL Blog", 3   "content": "This is my first blog created using the AI 4   BlogNest API with PostgreSQL.", 5   "category": "Technology" 6 }</pre> | <pre>{   "id": 1,   "title": "My First PostgreSQL Blog",   "content": "This is my first blog created using the AI   BlogNest API with PostgreSQL.",   "category": "Technology",   "author": 1,   "authorName": "Test User",   "createdAt": "2026-09-25T06:43:11.781Z",   "updatedAt": "2026-09-25T06:43:11.781Z" }</pre> |             |

The blog is stored in PostgreSQL.

## 5. Fetch All Blogs

**GET** http://localhost:5000/api/blogs — No authentication required.

|            |                                                                                                                                                                                                                                                                                                                                                                                            |             |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <b>GET</b> | http://localhost:5000/api/blogs                                                                                                                                                                                                                                                                                                                                                            | <b>Send</b> |
| Body       | Status: 200 OK Time: 92 ms Size: 587 B                                                                                                                                                                                                                                                                                                                                                     |             |
| (no body)  | <pre>[   {     "id": 1,     "title": "My First PostgreSQL Blog",     "content": "This is my first blog created using the   AI BlogNest API with PostgreSQL.",     "category": "Technology",     "author": {       "id": 1,       "name": "Test User",       "email": "testuser@gmail.com"     },     "authorName": "Test User",     "createdAt": "...",     "updatedAt": "..."   } ]</pre> |             |

## 6. Fetch Blog by ID

**GET** http://localhost:5000/api/blogs/1

|            |                                                                                                                                                                                                                                                                                                              |             |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <b>GET</b> | http://localhost:5000/api/blogs/1                                                                                                                                                                                                                                                                            | <b>Send</b> |
| Body       | Status: 200 OK Time: 55 ms Size: 541 B                                                                                                                                                                                                                                                                       |             |
| (no body)  | <pre>{   "id": 1,   "title": "My First PostgreSQL Blog",   "content": "This is my first blog created using the AI BlogNest API with PostgreSQL.",   "category": "Technology",   "author": {     "id": 1,     "name": "Test User",     "email": "testuser@gmail.com"   },   "authorName": "Test User" }</pre> |             |

The API returns the requested blog along with author information.

## 7. Update Blog

**PUT** http://localhost:5000/api/blogs/1 — Authorization: Bearer Token

|                                                                                                                                                                                       |                                                                                                                                                                                                                                 |             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <b>PUT</b>                                                                                                                                                                            | http://localhost:5000/api/blogs/1                                                                                                                                                                                               | <b>Send</b> |
| Body                                                                                                                                                                                  | Status: 200 OK Time: 61 ms Size: 512 B                                                                                                                                                                                          |             |
| <pre>1 { 2   "title": "My Updated PostgreSQL Blog", 3   "content": "This blog has been updated successfully using the AI BlogNest API.", 4   "category": "Data Analytics" 5 } 6</pre> | <pre>{   "id": 1,   "title": "My Updated PostgreSQL Blog",   "content": "This blog has been updated successfully using the AI BlogNest API.",   "category": "Data Analytics",   "updatedAt": "2026-09-25T07:10:02.331Z" }</pre> |             |

The `updated_at` timestamp is updated automatically.

## 8. Delete Blog

**DELETE** http://localhost:5000/api/blogs/1 — Authorization: Bearer Token

|               |                                                         |             |
|---------------|---------------------------------------------------------|-------------|
| <b>DELETE</b> | http://localhost:5000/api/blogs/1                       | <b>Send</b> |
| Body          | Status: 200 OK Time: 48 ms Size: 46 B                   |             |
| (no body)     | <pre>{   "message": "Blog deleted successfully" }</pre> |             |

## 9. Generate AI Blog

The AI BlogNest application integrates Google Gemini AI to automatically generate blog content.

**POST** http://localhost:5000/api/ai/generate-blog — Authorization: Bearer Token

|                                                                                                            |                                                                                                                                                                                                                                                                                                                                             |             |
|------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <b>POST</b>                                                                                                | http://localhost:5000/api/ai/generate-blog                                                                                                                                                                                                                                                                                                  | <b>Send</b> |
| Body                                                                                                       | Status: 201 Created Time: 4.12 s Size: 1.26 KB                                                                                                                                                                                                                                                                                              |             |
| <pre> 1 { 2   "topic": "Benefits of Data Analytics for Businesses", 3   "category": "Technology" 4 }</pre> | <pre> {   "id": 2,   "title": "Benefits of Data Analytics for Businesses",   "content": "Data analytics has become a cornerstone of modern business strategy, helping organizations uncover patterns, forecast trends and make evidence-based decisions...",   "category": "Technology",   "author": 1,   "authorName": "Test User" }</pre> |             |

The generated blog is automatically stored in the `blogs` table.

## Processing Flow

User Topic → AI Controller → Gemini Service → Google Gemini API → Generated Blog Content → PostgreSQL → JSON Response

## 10. Summarize Blog

The application also provides AI-powered summarization.

**POST** http://localhost:5000/api/ai/summarize

|                                                                                                                                                                                                                                                                                |                                                                                                                                                                                |             |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <b>POST</b>                                                                                                                                                                                                                                                                    | http://localhost:5000/api/ai/summarize                                                                                                                                         | <b>Send</b> |
| Body                                                                                                                                                                                                                                                                           | Status: 200 OK Time: 5.67 s Size: 702 B                                                                                                                                        |             |
| <pre> 1 { 2   "content": "Data analytics helps businesses make better decisions by using data to identify trends, understand customers, improve operations, and reduce risks." 3 } 4 5 </pre> <p>The exact generated summary may vary because it is produced by Gemini AI.</p> | <pre> {   "summary": "Data analytics helps businesses make better decisions by identifying trends, understanding customers, improving operations, and reducing risks." }</pre> |             |

The exact generated summary may vary because it is produced by Gemini AI.

## 11. Root API Test

**GET** http://localhost:5000/

|            |                                                           |             |
|------------|-----------------------------------------------------------|-------------|
| <b>GET</b> | http://localhost:5000/                                    | <b>Send</b> |
| Body       | Status: 200 OK Time: 12 ms Size: 46 B                     |             |
| (no body)  | <pre> {   "message": "AI BlogNest API is running" }</pre> |             |



## API Endpoint Summary

| Method | Endpoint              | Authentication | Purpose                   |
|--------|-----------------------|----------------|---------------------------|
| POST   | /api/auth/register    | No             | Register user             |
| POST   | /api/auth/login       | No             | Login user                |
| GET    | /api/auth/profile     | Yes            | Get authenticated profile |
| POST   | /api/blogs            | Yes            | Create blog               |
| GET    | /api/blogs            | No             | Get all blogs             |
| GET    | /api/blogs/:id        | No             | Get single blog           |
| PUT    | /api/blogs/:id        | Yes            | Update own blog           |
| DELETE | /api/blogs/:id        | Yes            | Delete own blog           |
| POST   | /api/ai/generate-blog | Yes            | Generate AI blog          |
| POST   | /api/ai/summarize     | No             | Summarize content         |
| GET    | /                     | No             | Check API status          |

## Security Implementation

### Password Hashing

Passwords are never stored as plain text. The application uses **bcryptjs** to hash passwords before storing them in PostgreSQL.

### JWT Authentication

JSON Web Tokens are used to authenticate users. A token is generated after successful login and must be supplied when accessing protected routes.

### SQL Parameterization

PostgreSQL queries use parameterized values, for example:

```
pool.query(
  'SELECT id FROM users WHERE email = $1',
  [email]
);
```

This helps prevent SQL injection by separating SQL commands from user-supplied values.

### Request Validation

express-validator validates incoming request data before controller processing.

### Error Handling

The project uses centralized error handling. The middleware `src/middleware/errorMiddleware.js` returns JSON error responses, for example:

```
{ "message": "Internal Server Error" }
```

Validation errors are returned with HTTP status 422. Authentication errors use appropriate 401 responses. Authorization failures use 403 responses.

## AI Integration Architecture

The AI functionality is separated from the main controller logic.

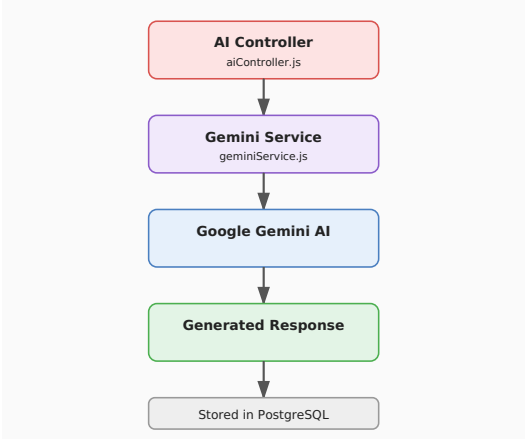


Fig 5.1 — AI integration architecture

This separation keeps external AI communication independent from the API routing and database logic.

### Database Migration

The original project was configured for MongoDB/Mongoose. During development, the database layer was migrated to PostgreSQL.

| Original                     | Final Implementation                      |
|------------------------------|-------------------------------------------|
| Node.js → Mongoose → MongoDB | Node.js → pg / node-postgres → PostgreSQL |

The final application uses **PostgreSQL** as its active database. The MongoDB/Mongoose models were removed from the active application code, and the application was tested successfully using PostgreSQL.

### Final Project Testing Status

The following components were tested successfully:

| Component                 | Status  |
|---------------------------|---------|
| Node.js server            | Working |
| Express API               | Working |
| PostgreSQL connection     | Working |
| User registration         | Tested  |
| User login                | Tested  |
| JWT authentication        | Tested  |
| User profile              | Tested  |
| Create blog               | Tested  |
| Get all blogs             | Tested  |
| Get blog by ID            | Tested  |
| Update blog               | Tested  |
| Delete blog               | Tested  |
| Gemini AI blog generation | Tested  |
| Gemini AI summarization   | Tested  |
| Root API endpoint         | Tested  |

### Conclusion

The AI BlogNest project successfully implements a modular and secure backend REST API for AI-assisted blog management. The application combines Node.js, Express.js, PostgreSQL, JWT authentication, bcrypt.js, and Google Gemini AI to provide user authentication, blog CRUD operations, AI-powered blog generation, and content summarization. The PostgreSQL database provides structured and relational data storage for users and blogs, while the authentication middleware

protects user-specific operations.

The Gemini AI integration extends the traditional blog management system by automatically generating and summarizing content. The project was successfully executed and tested through Postman, with the major API endpoints and database operations functioning correctly.